



Prática: threads em C/Python

Aula 16 de 60 — Sistemas Operacionais

Conceito

Laboratório prático de programação concorrente. Criação de threads com `pthread_create` (função, argumento `void*`, retorna `pthread_t`). Sincronização com mutex (`pthread_mutex_lock/unlock`) e semáforos (`sem_wait/sem_post`). Problemas: soma paralela de vetor (divisão entre N threads), produtor-consumidor com buffer circular. Depuração com gdb (thread apply all bt para backtrace de todas threads). Análise com helgrind (Valgrind) detecta race conditions. Desempenho: $\text{speedup} = \text{tempo_sequencial} / \text{tempo_paralelo}$, limitado pela Lei de Amdahl.

Pratica: Threads

`pthread_create / pthread_join`

Mutex lock/unlock

Semaforos

Valgrind helgrind / gdb

Lei de Amdahl: speedup limitado

Atividade

Implemente em C soma paralela de vetor de 10 milhões de inteiros dividindo entre 4 threads. Compare com versão sequencial (`clock_gettime`). Espera-se speedup entre 2x e 3x dado overhead de criação e sincronização.

Pesquisa

Pesquise um bug histórico de race condition: Therac-25 (1985) ou bug do DNS NTP (2014, `getrandom` causou timeout em massa). Explique causa raiz e correção.



Requisitos

- `gcc`
- `linux`
- `valgrind`